1 Social Optimum: Diamond Network

1.1 Network

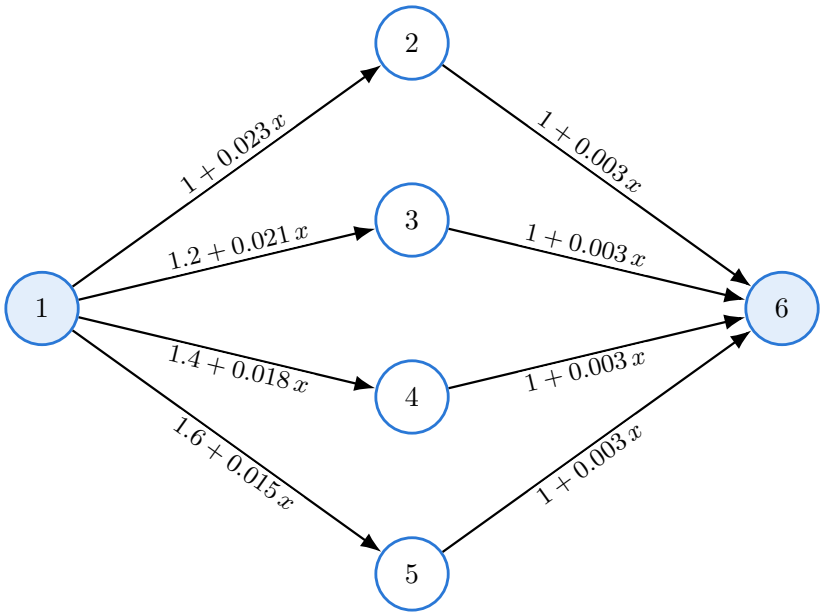


Figure 1: Diamond network. x is the number of agents on an edge in a timestep.

All agents travel $1 \to 6$ on one of four disjoint routes $R_k = 1 \to k \to 6$. Adding its two edges, R_k costs $A_k + B_k n$ per agent with n agents on it:

	R_2	R_3	R_4	R_5
A_k	2.0	2.2	2.4	2.6
B_k	0.026	0.024	0.021	0.018

Cheaper routes congest faster. Every route's edges are crossed at steps 0 and 1, so timing plays no role.

1.2 Derivation

With n_k agents on R_k,

$$SC = \sum_k (A_k n_k + B_k n_k^2), \qquad \sum_k n_k = N.$$

At the optimum every used route has the same marginal cost λ:

$$A_k + 2B_k n_k = \lambda \quad \Longrightarrow \quad n_k = \frac{\lambda - A_k}{2B_k},$$

with λ chosen so that $\sum_k n_k = N$. A route is used only if $A_k < \lambda$, so routes open cheapest first as N grows; R_5 needs $N > 24.6$.

Whole agents. The j-th agent on R_k adds $A_k + B_k(2j-1)$, which grows with j. So adding agents one at a time to the cheapest route is exactly optimal.

1.3 Results

N	n_2	n_3	n_4	n_5	λ	SC_{opt}	SC_{opt}/N
20	10	7	3	0	2.53	46.565	2.32825
50	17	14	11	8	2.88	127.911	2.55822
80	23	21	19	17	3.20	219.121	2.73901

Table 1: Optimal number of agents on each route.

- $N = 20$: $\lambda \approx 2.53 < 2.6$, so R_5 stays empty.
- With fractional agents, each agent pays $(A_k + \lambda)/2$, so each cheaper route's users pay exactly 0.1 less: the optimum deliberately loads cheap routes more.
- SC_{opt}/N rises from 2.33 to 2.74 as N grows from 20 to 80.